

UNIVERSITÀ DEGLI STUDI DI NAPOLI FEDERICO II

SCUOLA POLITECNICA E DELLE SCIENZE DI BASE

DIPARTIMENTO DI INGEGNERIA ELETTRICA E TECNOLOGIE
DELL'INFORMAZIONE

CORSO DI LAUREA TRIENNALE IN INFORMATICA

A FANCY L^AT_EX TEMPLATE FOR YOUR THESIS
AT UNINA

Relatori

Megadir. Prof. Guidobaldo Maria

RICCARDELLI

Duca Conte Francesco BARAMBANI

Baronessa Filiguelli DE BONCHAMPS

Correlatore

Megadir. Clam. Duca Conte Pier

Carlo ing. SEMENZARA

Candidato

Marco FESTA

N86004979

Anno Accademico 2025–2026

Sommario

Il presente lavoro di tesi illustra le attività di ricerca e sviluppo condotte durante il periodo di tirocinio presso **InfoCube**, focalizzandosi sull'estrazione, strutturazione e interrogazione semantica avanzata di documenti legali e normativi. Il progetto si inserisce nel contesto più ampio di **LUDOVICA** (*Language Understanding for Document Optimization and Validation In Civil Administration*), un motore generativo basato sull'Intelligenza Artificiale volto a supportare la pubblica amministrazione locale nell'elaborazione e stesura di atti amministrativi.

Il problema tecnico principale affrontato in questo lavoro è la trasformazione di documenti PDF complessi in una base di conoscenza solida e interrogabile dalle macchine (**Knowledge Graph**). A tale scopo, la rigida gerarchia tipica degli atti normativi (Titoli, Capi, Articoli, Commi) è stata modellata e salvata all'interno del database a grafo **Neo4j**[1], permettendo di preservare fedelmente le relazioni strutturali e concettuali dei testi originali.

Per abilitare il recupero intelligente e contestuale delle informazioni, è stato implementato un sistema di **Semantic Search**. Questa fase ha richiesto il testing di molteplici **modelli di embedding**, allo scopo di individuare la soluzione più performante per la vettorializzazione del complesso gergo giuridico italiano. Al fine di validare le scelte architetturali, è stata condotta un'analisi comparativa diretta tra le performance di un approccio di ricerca vettoriale classica, basata sulla *cosine similarity*, e i vantaggi offerti dalla navigazione topologica e strutturale nel database a grafo.

SCRIVERE QUI I RISULTATI OTTENUTI, LE CONCLUSIONI E EVENTUALI PROSPETTIVE FUTURE.

Indice

1	Introduzione	1
1.1	Definizione del problema	1
1.2	Obiettivi del lavoro di tesi	2
1.3	Descrizione dell’approccio	2
1.4	Confronto con tecniche esistenti	2
1.5	Struttura della tesi	3
2	Tecnologie utilizzate	4
2.1	Linguaggi di Programmazione	4
2.2	Strumenti per l’estrazione del Testo	4
2.3	Scelta del Database	5
2.4	LLM e Modelli di Embedding Semantici	6
2.5	Deployment e Containerizzazione	7
3	Architettura del Sistema	9
4	Implementazione Software	11
4.1	Estrazione dei Documenti	11
4.1.1	Open Data Loader e la Gestione dei Tagged PDF	11
4.1.2	Docling e l’Integrazione dei Motori OCR	12
4.1.3	Progettazione del Custom Triage Processor	13
4.2	Strutturazione dei Documenti tramite LLM	15
4.2.1	Il Modello e la Strategia di Generazione	15
4.2.2	Estrazione delle Relazioni Semantiche	16
4.3	Importazione dati nei Database	17
4.3.1	I Modelli di Dominio	17
4.3.2	Interfaccia per il disaccoppiamento della persistenza	18
4.3.3	Architettura del Database Neo4j	19
4.3.4	Dettagli Implementativi dell’Ingestion su Neo4j	20
4.3.5	Risultati e Metriche	22
4.3.6	Visualizzazione del Grafo	23
4.4	Creazione degli Embedding	24

4.4.1	Differenza nell'Arricchimento Contestuale: Standard vs Late Chunks	24
4.4.2	Il Graph Embedding come appoggio agli Embedding Semantici .	25
Bibliografia		27

Elenco delle figure

3.1	Architettura del sistema	10
4.1	Flowchart del Custom Triage Processor	14
4.2	Class Diagram dei modelli di dominio	18
4.3	Rappresentazione visiva delle relazioni instaurate tra i nodi dell'articolo 8 di un atto normativo.	21
4.4	Rappresentazione visiva di un estratto del database a grafo Neo4j.	23

Elenco delle tabelle

4.1	Statistica dei nodi inseriti nel database Neo4j.	22
4.2	Statistica delle relazioni inserite nel database Neo4j.	22

-1-

Introduzione

CONTENTS: 1.1 Definizione del problema. 1.2 Obiettivi del lavoro di tesi. 1.3 Descrizione dell'approccio. 1.4 Confronto con tecniche esistenti. 1.5 Struttura della tesi.

1.1 Definizione del problema

La pubblica amministrazione locale italiana fa quotidianamente affidamento su una vasta e complessa mole di testi normativi, atti e delibere. L'elaborazione automatica di tali documenti è tuttavia ostacolata da diverse sfide intrinseche: i testi sono spesso archiviati in formati non strutturati (come i PDF), presentano difformità di impaginazione, impiegano un linguaggio burocratico peculiare e si articolano in una rigida e profonda struttura gerarchica. Quando si tenta di automatizzare la ricerca e la stesura di nuovi atti attraverso sistemi basati su Intelligenza Artificiale Generativa, l'approccio classico - basato unicamente sul recupero esplorativo di porzioni di testo destrutturato (frammenti di testo o "chunk") - si rivela insufficiente, portando inevitabilmente alla potenziale perdita del contesto legale. È in questo scenario che si inserisce il lavoro di tesi, svolto nell'ambito delle attività di tirocinio presso **InfoCube** per contribuire allo sviluppo del progetto **LUDOVICA**, il cui obiettivo finale è fornire ai funzionari della PA un motore generativo affidabile, capace di recuperare gli atti pregressi pertinenti e generare template documentali pronti all'uso. Affinché le risposte del sistema siano rigorose e verificabili è emersa l'esigenza di adottare un'architettura dati che introduca una solida topologia informativa.

Il problema tecnico fondamentale affrontato consiste quindi nella necessità di **convertire i documenti normativi complessi in una base di conoscenza interrogabile** (*Knowledge Graph*) che conservi fedelmente la struttura semantica e gerarchica originale.

1.2 Obiettivi del lavoro di tesi

Al fine di superare i limiti esposti e dotare il sistema di solidi meccanismi per l'interpretazione del linguaggio giuridico, il lavoro si è articolato nei seguenti obiettivi:

1. **Modellazione topologica e digitalizzazione:** Costruire una pipeline di inserimento per tradurre documenti legali non strutturati in un database a grafo (*Neo4j*), definendo rigorosamente la mappatura delle relazioni.
2. **Semantic Search per il dominio giuridico:** Sviluppare il modulo di interrogazione intelligente del database e quantificare, mediante il testing di molteplici modelli di embedding, quale sia la soluzione più performante nel catturare la semantica intrinseca del linguaggio legale in lingua italiana.
3. **Analisi prestazionale degli approcci di Retrieval:** Eseguire un confronto diretto per misurare la differenza qualitativa delle risposte e confrontare le prestazioni fornite da un approccio di ricerca puramente vettoriale, basato sulla *cosine similarity*, rispetto a un'interrogazione che sfrutti attivamente la navigazione topologica sul grafo.

1.3 Descrizione dell'approccio

Per raggiungere gli obiettivi prefissati, è stata ideata un'architettura di elaborazione in grado di trasformare documenti non strutturati in una rete di nodi interconnessi. L'approccio proposto si articola in una pipeline a più fasi: inizialmente, i documenti originali (principalmente in formato PDF) vengono processati tramite strumenti di parsing avanzati e OCR per estrarne accuratamente il testo e convertirlo in un formato intermedio (Markdown). Successivamente, un *Large Language Model* analizza tale testo per riconoscerne la suddivisione gerarchica (Titoli, Capi, Articoli, Commi) e le relazioni intrinseche tipiche della struttura giuridica (ad esempio un comma che cita un altro comma). Questi elementi vengono strutturati in formato JSON e iniettati all'interno del grafo, in cui ogni entità diventa un nodo e l'appartenenza a una data sezione diventa una relazione. Successivamente, i contenuti testuali vengono vettorializzati e salvati sotto forma di *embedding*. In fase di interrogazione, il sistema non si limita a cercare la similarità matematica tra le frasi, ma naviga esplicitamente la topologia del grafo per ricostruire l'intero iter contestuale, fornendo risposte documentate, ricche e strutturalmente coerenti.

1.4 Confronto con tecniche esistenti

Nel panorama attuale dell'Intelligenza Artificiale applicata all'elaborazione dei documenti, le tecniche di *Information Retrieval* più diffuse si basano principalmente su *Retrieval-Augmented Generation* (RAG) di tipo vettoriale. Quest'ultima prevede tipicamente la

suddivisione del documento in frammenti testuali di lunghezza fissa (*chunking*), la loro vettorializzazione e il successivo recupero basato sulla distanza semantica rispetto alla query dell'utente.

Tuttavia, queste tecniche presentano lacune significative quando applicate al dominio giuridico della pubblica amministrazione. Un approccio RAG standard, estraendo un singolo *chunk* isolato, perde inevitabilmente il perimetro normativo: ad esempio, potrebbe recuperare il comma di una legge senza specificare a quale Capo appartenga oppure omettendo dettagli cruciali come eventuali eccezioni specificate in altri paragrafi. L'approccio ibrido basato su *Knowledge Graph* (spesso definito *GraphRAG*) si rende pertanto strettamente necessario. Modellando esplicitamente le relazioni tra i componenti di una legge, esso supera i limiti del chunking piatto, consentendo di restituire un contesto normativo integro e affidabile. Questo aspetto è fondamentale per garantire l'assoluta affidabilità formale e legale delle risposte, un requisito imprescindibile per un software destinato agli operatori della PA.

1.5 Struttura della tesi

Il presente elaborato è organizzato in sei capitoli, di seguito brevemente introdotti.

Il Capitolo 2 — Tecnologie utilizzate fornisce una panoramica dello stack tecnologico adottato: dai linguaggi di programmazione agli strumenti di parsing e OCR per l'estrazione del testo, passando per il database a grafo, fino ai modelli di linguaggio e di embedding impiegati per la strutturazione semantica e la vettorializzazione dei contenuti normativi.

Il Capitolo 3 — Architettura del Sistema illustra l'architettura complessiva del sistema, descrivendo i componenti principali e il flusso di dati attraverso le diverse fasi della pipeline: dall'acquisizione dei documenti grezzi fino all'interrogazione da parte degli utenti finali.

Il Capitolo 4 — Implementazione Software approfondisce nel dettaglio le singole fasi della pipeline di elaborazione: l'estrazione del testo dai documenti PDF, la strutturazione gerarchica operata dai Large Language Models, l'ingestion dei dati strutturati su Neo4j e la creazione degli embedding per la ricerca semantica.

Il Capitolo 5 — Ricerca Semantica Vettoriale presenta il setup sperimentale, le metriche di valutazione adottate e i risultati ottenuti con l'approccio di ricerca puramente vettoriale basato sulla *cosine similarity*. (DA SCRIVERE)

Il Capitolo 6 — Ricerca Semantica con Grafo analizza i risultati del confronto tra l'approccio basato sul grafo e quello puramente vettoriale, evidenziando le differenze in termini di performance ed efficacia nella restituzione del contesto normativo. (DA SCRIVERE)

–2–

Tecnologie utilizzate

Per far fronte alle sfide di estrazione, elaborazione, archiviazione e interrogazione dei documenti normativi, è stato impiegato uno stack tecnologico che combina strumenti di parsing avanzato, modelli di LLM ed embedding e database specializzati. Di seguito vengono presentate le tecnologie adottate, illustrando per ciascuna le motivazioni che ne hanno guidato la scelta.

2.1 Linguaggi di Programmazione

- **Python**[2]: È stato il linguaggio cardine per la costruzione dell'intera pipeline del progetto. La scelta è ricaduta su Python per diverse ragioni:
 - **Ecosistema dedicato ai modelli linguistici:** Python dispone di librerie mature e ampiamente supportate (come `sentence-transformers`) che hanno permesso di integrare agevolmente i modelli di embedding senza dover reimplementare componenti di basso livello.
 - **Ottima integrazione con tutte le librerie utilizzate:** Python risulta perfettamente integrato con tutti gli strumenti e le librerie adottate nell'intero stack tecnologico (dai tool di estrazione testuale, ai modelli LLM, fino alla Graph Data Science Library e al database Neo4j tramite API native), permettendo di orchestrare l'intera pipeline in modo fluido e senza la necessità di complessi adattatori o wrapper di terze parti.

2.2 Strumenti per l'estrazione del Testo

La fase di estrazione ha dovuto confrontarsi con documenti PDF eterogenei: alcuni nativamente digitali e ben strutturati, altri derivanti da scansioni cartacee di qualità variabile. Per questo motivo è stata adottata una combinazione di strumenti complementari.

- **Open Data Loader**[3]: Libreria open-source per l'estrazione e il caricamento di dati da fonti eterogenee. È stata selezionata per due caratteristiche fondamentali:
 - **Determinismo**: A differenza di approcci basati su modelli probabilistici, Open Data Loader estrae il testo in modo deterministico, garantendo coerenza dei risultati, un requisito indispensabile nel dominio legale, dove ogni modifica del testo estratto tra un'esecuzione e l'altra minerebbe la fiducia nel sistema.
 - **Conversione diretta in Markdown**: Il Markdown si rivela un formato testuale ottimale per i LLM. La sua sintassi leggera preserva fedelmente la struttura logica del documento originale, fornendo al modello un contesto già strutturato che riduce significativamente gli errori nella successiva fase di parsing gerarchico.
- **Docling**[4]: Libreria specializzata nell'analisi del layout documentale e nell'estrazione di contenuti strutturati. È stata integrata come supporto complementare a Open Data Loader per la gestione di PDF più complessi: supporta documenti multi-colonna, tabelle e layout non lineari, frequenti negli atti della pubblica amministrazione.
- **RapidOCR**[5]: Motore di *Optical Character Recognition* impiegato per digitalizzare i documenti derivanti da scansioni visive. La scelta è stata guidata da un'analisi comparativa tra diversi motori OCR:
 1. **EasyOCR**: Configurato di default all'interno di Docling, adotta modelli basati su Deep Learning altamente accurati ma estremamente esigenti in termini computazionali. Anche isolando l'elaborazione sulla singola pagina, il consumo di RAM era così elevato che rendeva impossibile l'utilizzo nell'ambiente di produzione containerizzato.
 2. **Tesseract OCR**: Si è dimostrato estremamente leggero e pienamente compatibile con i limiti di memoria imposti, scongiurando qualsiasi crash del container. Tuttavia, nel contesto specifico del linguaggio legale italiano, il motore non è stato in grado di interpretare correttamente il layout, fallendo l'estrazione del testo e restituendo output incompleti o disallineati rispetto alla struttura originale del documento.
 3. **RapidOCR**: Questo specifico modulo è stato infine selezionato per l'estrazione del testo dalle scansioni visive. RapidOCR si è rivelato il compromesso ideale, offrendo un perfetto bilanciamento tra l'accuratezza del riconoscimento del testo italiano e il consumo di RAM del container.

2.3 Scelta del Database

La duplice natura del problema — da un lato la necessità di rappresentare relazioni gerarchiche complesse, dall'altro l'esigenza di una ricerca semantica efficiente — ha

portato all'adozione di un database a grafo specializzato. La scelta è ricaduta su Neo4j, il database a grafo più diffuso e maturo sul mercato, per le seguenti motivazioni:

- **Modellazione naturale della gerarchia normativa:** La struttura di un atto normativo è intrinsecamente un grafo, in cui ogni elemento è un nodo e le relazioni di appartenenza sono archi. Neo4j consente di mappare questa topologia in modo diretto, senza le complessità delle JOIN ricorsive tipiche dei database relazionali.
- **Supporto nativo per la Vector Search:** Il database permette di memorizzare vettori ad alta dimensionalità (fino a 4096 dimensioni) come proprietà dei nodi. Questa caratteristica, unita alla possibilità di creare indici vettoriali dedicati, risulta fondamentale per le operazioni di ricerca basate sulla *cosine similarity*.

Per interagire con il database e gestire le operazioni di interrogazione e popolamento del grafo, è stato adottato **Cypher** [6], il linguaggio dichiarativo nativo dell'ecosistema Neo4j.

Infine, per valorizzare le informazioni topologiche della rete, si è fatto ricorso alla **Graph Data Science Library** [7], una libreria nativa dedicata all'implementazione di algoritmi analitici sui grafi. Al suo interno è stato selezionato **Node2Vec** [8], un potente algoritmo di *graph embedding*. A differenza degli embedding semantici utilizzati per la ricerca testuale (discussi nella Sezione 2.4), questo algoritmo permette di codificare in uno spazio vettoriale di 128 dimensioni le complesse relazioni gerarchiche e di vicinato che intercorrono tra i vari elementi del corpus normativo.

2.4 LLM e Modelli di Embedding Semantici

Il cuore intelligente del sistema risiede nei modelli di linguaggio e di embedding, responsabili rispettivamente della strutturazione semantica dei documenti e della loro vettorializzazione per la ricerca.

- **Gemini 3.5 Flash**[9]: *Large Language Model* di Google, utilizzato tramite API per il compito cruciale della strutturazione dei documenti normativi. Le motivazioni alla base di questa scelta sono:
 - **Aderenza rigorosa allo schema JSON:** Gemini 3.5 Flash ha dimostrato un'eccellente capacità di produrre output JSON conformi a uno schema predefinito, un requisito fondamentale per garantire che la gerarchia normativa (Titoli, Capi, Articoli, Commi) fosse rappresentata in modo consistente e automaticamente processabile dalle fasi successive della pipeline.
 - **Comprensione della lingua italiana:** Il modello ha mostrato una solida competenza nell'elaborazione del linguaggio burocratico italiano, riconoscendo correttamente la terminologia giuridica e le espressioni formulari tipiche degli atti amministrativi.

- **Rapporto costo/qualità:** Rispetto a modelli più potenti ma significativamente più costosi (come GPT-4 o Claude), Gemini 3.5 Flash ha offerto un compromesso ottimale tra qualità dell'output strutturato e costo per token, aspetto rilevante considerando l'elaborazione batch di centinaia di documenti.
- **Modelli di Embedding:** La vettorializzazione del testo normativo rappresenta un elemento critico per la qualità della ricerca semantica. Sono stati testati molteplici modelli per identificare la soluzione più efficace nel dominio giuridico italiano. I test più approfonditi hanno coinvolto i seguenti modelli:
 - **google/embeddinggemma-300m:** Modello sviluppato da Google che trasforma il testo in vettori a 768 dimensioni. È costruito sull'architettura Gemma 3 ed è addestrato su oltre 100 lingue. Una caratteristica distintiva è la possibilità di ridurre le dimensioni del vettore finale (a 512, 256 o 128) senza perdere qualità, rendendolo particolarmente efficiente su dispositivi con risorse limitate.
 - **perplexity-ai/pplx-embed-v1-0.6b:** Modello sviluppato da Perplexity AI che trasforma il testo in vettori a 1024 dimensioni, maggiori rispetto a Gemma per catturare più sfumature semantiche. È ottimizzato per applicazioni pratiche come ricerca semantica e sistemi RAG.
 - **perplexity-ai/pplx-embed-context-v1-0.6b:** Variante contestuale del modello Perplexity, anch'esso produce vettori a 1024 dimensioni. La differenza principale è che questo modello è addestrato per comprendere il *contesto* di un documento: quando processa i chunk, considera anche il resto del documento, non solo il frammento isolato. Questo lo rende più adatto per sistemi RAG dove il significato dei frammenti dipende dal contesto circostante. Questa tecnica è nota come *Late Chunking* [10] e permette di ottenere rappresentazioni più ricche e accurate dei frammenti di testo.

La valutazione si è basata sulla capacità di ciascun modello di recuperare i commi pertinenti a partire da query in linguaggio naturale, misurata attraverso metriche standard di Information Retrieval quali *Precision@k*, e *Mean Reciprocal Rank* (MRR).

2.5 Deployment e Containerizzazione

- **Docker**[11]: L'intero ecosistema è stato modularizzato attraverso lo sviluppo di container isolati e specifici per ciascun componente della pipeline. Questa scelta architetturale è stata dettata da esigenze sia tecniche che operative:
 - **Isolamento delle dipendenze:** Ogni fase della pipeline (estrazione, strutturazione, ingestion, embedding) presenta dipendenze software potenzialmente conflittuali. Docker ha permesso di incapsulare ciascuna in un ambiente dedicato, eliminando i conflitti di versione tra librerie e garantendo la riproducibilità dell'ambiente di esecuzione su qualsiasi macchina.

- **Modularità e manutenibilità:** La separazione in container indipendenti ha permesso di sviluppare, testare e aggiornare ciascun modulo della pipeline in isolamento, semplificando la manutenzione.
- **Scalabilità orizzontale:** L'architettura a container costituisce un prerequisito naturale per futuri scenari di deployment distribuito in cui diverse istanze della pipeline possano operare concorrentemente su lotti distinti di documenti.

–3–

Architettura del Sistema

In questo capitolo viene presentata l'architettura complessiva del sistema sviluppato, illustrando i componenti principali e il flusso di dati attraverso le diverse fasi del processo. L'obiettivo è fornire una panoramica chiara e dettagliata delle scelte progettuali, tracciando il percorso logico che i dati seguono dal momento in cui vengono acquisiti fino alla fase di interrogazione e utilizzo da parte degli utenti finali.

La Figura 3.1 fornisce un diagramma riassuntivo di tale architettura, evidenziando i moduli principali e le interazioni tra di essi. Come illustrato nello schema, il sistema è concepito come una pipeline sequenziale.

Nella prima fase, vengono applicate tecniche specifiche per il recupero dei testi grezzi dai documenti normativi, con particolare attenzione alle librerie impiegate per il parsing e alle strategie di pulizia dei dati, fondamentali per mitigare rumore e inconsistenze strutturali.

Una volta estratto, il testo necessita di essere organizzato secondo le logiche giuridiche del documento. A tale scopo, la fase di strutturazione si avvale di Large Language Models (LLM) per la trasformazione del testo grezzo in un formato analizzabile. Il fulcro di questo passaggio è la definizione e l'utilizzo di uno schema JSON rigoroso, progettato appositamente per rappresentare e mantenere intatta la gerarchia degli atti normativi e le complesse relazioni giuridico-logiche che li collegano.

I dati validati e strutturati vengono successivamente indirizzati verso il sistema di archiviazione, cioè il database Neo4j, che inserisce le informazioni mantenendo le relazioni gerarchiche e logiche tra i vari elementi normativi precedentemente individuati.

Dopodiché vengono calcolati gli *embedding* testuali a partire dal testo dei commi, informazione presente come proprietà del nodo. Questi *embedding* vengono memorizzati nel database, consentendo di effettuare ricerche basate sulla similarità semantica.

Infine viene effettuato un confronto architetturale e prestazionale tra due diversi paradigmi di ricerca:

- L'implementazione di una **ricerca vettoriale classica** basata puramente sulla *cosine similarity*. Sfruttando le rappresentazioni vettoriali archiviate, questo approccio è mirato a simulare la ricerca per similarità in un ambiente vettoriale puro, fungendo

così da termine di paragone fondamentale per la successiva fase di benchmarking tra l'interrogazione puramente semantica e quella arricchita dalla struttura a grafo.

- L'implementazione di una **ricerca ibrida (vettoriale e strutturale)**, in cui il calcolo della similarità semantica viene arricchito dalla topologia del grafo. In questa configurazione, il database non si limita a ospitare gli *embedding* testuali, ma sfrutta le relazioni gerarchiche, sequenziali e logiche del corpus normativo. Questo approccio permette di combinare il recupero dell'informazione semantica con il contesto relazionale fornito dalla struttura del documento.

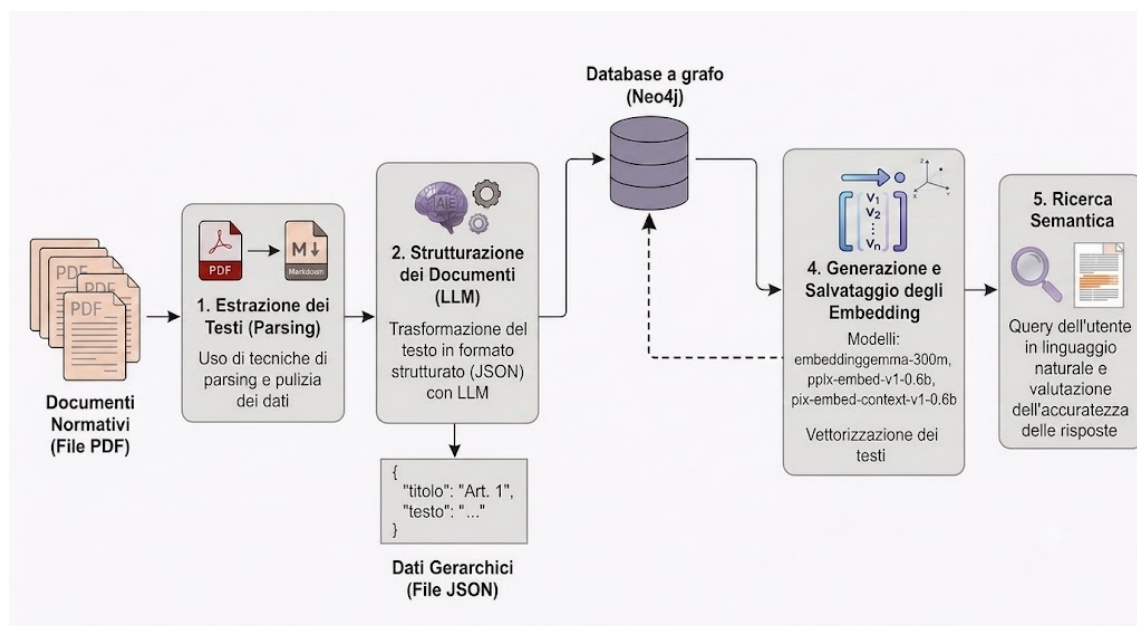


Figura 3.1: Architettura del sistema

–4–

Implementazione Software

CONTENTS: 4.1 Estrazione dei Documenti. 4.2 Strutturazione dei Documenti tramite LLM. 4.3 Importazione dati nei Database. 4.4 Creazione degli Embedding.

In questo capitolo verranno analizzate nel dettaglio le singole fasi della pipeline di elaborazione, partendo dalle strategie di estrazione del testo grezzo, passando per la strutturazione operata dai Large Language Models, fino ad arrivare alle logiche di ingestion e interrogazione sui database

4.1 Estrazione dei Documenti

La fase iniziale della pipeline prevede l'estrazione del testo dai documenti normativi in formato PDF.

Nelle prime fasi di sviluppo del progetto, l'intero documento PDF veniva passato direttamente al Large Language Model, demandando a quest'ultimo sia il compito di estrazione del testo grezzo sia quello di strutturazione sintattica. Questo approccio presentava forti criticità, esponeva il sistema al rischio di allucinazioni ed era inefficiente nel parsing di strutture complesse. Per superare tali limitazioni, la pipeline è stata modificata separando nettamente l'estrazione del testo dalla sua successiva strutturazione.

4.1.1 Open Data Loader e la Gestione dei Tagged PDF

Per l'estrazione del testo da documenti digitali nativi è stata adottata la libreria open-source *Open Data Loader*. Questo strumento converte localmente i file PDF in formati ottimizzati per i LLM, specificamente Markdown. Il vantaggio di questa soluzione risiede nella sua natura deterministica: a parità di input viene garantito il medesimo output, azzerando le variabilità tipiche dei modelli generativi.

Durante l'analisi dei documenti legali, una delle criticità maggiori riguarda i ritorni a capo spuri, generati per ragioni puramente stilistiche (ad esempio, il raggiungimento del

marginale della pagina). Se non gestiti correttamente, questi si traducono in doppi ritorni a capo nel testo Markdown finale, frammentando artificialmente i periodi e compromettendo la qualità della successiva fase di chunking. Per risolvere questa criticità alla radice, l'estrazione è stata configurata abilitando il parametro specifico `use_struct_tree=True`.

Quando questo parametro è attivo, la libreria modifica il suo approccio di lettura, verificando innanzitutto se il file analizzato appartiene alla categoria dei *Tagged PDF*. A differenza dei PDF standard, che definiscono esclusivamente il layout bidimensionale e visivo, i Tagged PDF incorporano un livello gerarchico logico e invisibile: il *Logical Structure Tree*. Questo albero organizza il contenuto tramite elementi strutturali del tutto simili ai tag HTML (come `<H1>`, `<P>`, `<Table>`, `<L>`).

Di conseguenza, abilitando `use_struct_tree=True`, *Open Data Loader* è in grado di ignorare le fallaci euristiche geometriche e navigare direttamente la struttura logica del documento. Questo meccanismo permette all'estrattore di mappare ogni porzione di testo discriminando con precisione assoluta i semplici a capo visivi dalle reali interruzioni di paragrafo, restituendo un flusso di testo continuo e fedele all'originale.

Nel contesto di questa tesi, focalizzata sugli atti della pubblica amministrazione locale, questa scelta tecnica si è rivelata non solo efficace, ma strategica. Le istituzioni pubbliche sono infatti vincolate da stringenti obblighi normativi sull'accessibilità digitale, che impongono la piena fruibilità dei documenti tramite tecnologie assistive, come gli screen reader. Per rispettare tali standard legali, i documenti pubblicati devono obbligatoriamente contenere un *Logical Structure Tree* rigoroso e ben formato. Pertanto, quello che per l'ente pubblico rappresenta un adempimento normativo inderogabile, si trasforma per la nostra pipeline di estrazione in una preziosa risorsa: una fonte di dati pre-strutturata in cui le gerarchie e l'ordine di lettura sono già nativamente validati alla radice.

4.1.2 Docling e l'Integrazione dei Motori OCR

L'approccio standard basato su *Open Data Loader* mostra evidenti limiti in presenza di documenti non strutturati, con layout complessi o derivanti da scansioni visive, in quanto tale libreria non integra nativamente alcun modulo OCR. Per sopperire a questa mancanza è stata introdotta nel sistema la libreria *Docling*.

A differenza di *Open Data Loader*, *Docling* supporta l'integrazione diretta di motori OCR, configurandosi come un sistema avanzato di parsing e analisi del layout. Questo strumento, specializzato nell'estrazione di metadati, tabelle e testo da formati eterogenei, è stato quindi affiancato a *Open Data Loader* all'interno della pipeline, intervenendo come supporto specifico per processare tutti quei documenti in cui la sola estrazione con *Open Data Loader* risulta impossibile o inefficace.

L'introduzione dell'OCR ha sollevato complessità non trascurabili relative alla gestione delle risorse hardware all'interno dell'architettura a container. Il container Docker deputato all'estrazione operava infatti in condizioni in cui non poteva sprecare troppa RAM, date le risorse hardware limitate. Durante i primi test, l'invio massivo di interi documenti scansionati causava la saturazione istantanea della memoria, provocando l'intervento del modulo del kernel Linux (*Out-Of-Memory Killer*) che terminava forzatamente il container.

Per superare il problema del consumo di RAM e prevenire i crash di sistema, è stata implementata una strategia basata sull'elaborazione pagina-per-pagina. Come discusso nel Capitolo 2 relativo alle scelte tecnologiche, a seguito di un'analisi comparativa dei motori supportati da Docling, la scelta è ricaduta su **RapidOCR**. Questo motore ha permesso di mantenere il consumo di memoria adeguato per il funzionamento del container, garantendo l'accuratezza del riconoscimento sul testo legale italiano.

4.1.3 Progettazione del Custom Triage Processor

Il processore di triage nativo di Open Data Loader prevede un flusso standard in cui ogni pagina viene analizzata e smistata: le pagine semplici vengono processate tramite il processo di Open Data Loader (una JVM), mentre le pagine complesse vengono delegate al backend basato su Docling.

Tuttavia, l'applicazione con l'uso del parametro `use_struct_tree=True` introduce un bug logico nel triage di default: i documenti non espressamente *tagged* vengono forzati all'interno del processo standard di Open Data Loader, non venendo mai instradati verso il backend di Docling, escludendo così l'analisi per pagine troppo complesse oppure l'attivazione dell'OCR dove necessario.

Per risolvere questa problematica è stato progettato e implementato un **Custom Triage Processor**. Come illustrato nel diagramma di flusso in **Figura 4.1**, il componente corregge le anomalie di instradamento agendo secondo una logica a tre fasi:

- **Fase 1: Verifica preliminare della struttura.** Il file PDF viene acquisito e il sistema ne analizza i metadati per verificare se è codificato come *Tagged PDF*. In caso affermativo, l'intero documento viene elaborato direttamente da Open Data Loader sfruttando l'albero strutturale nativo. Questo approccio garantisce la massima velocità di esecuzione e una pulizia ottimale del testo estratto.
- **Fase 2: Frazionamento del documento e stima della complessità.** Se il documento risulta privo di struttura (*Non-Tagged*), il flusso prevede la divisione in singole pagine indipendenti. Questa separazione è fondamentale per prevenire sovraccarichi di memoria RAM durante le elaborazioni successive. Per ciascuna pagina, il sistema calcola poi la densità dei caratteri testuali per valutarne il grado di complessità.
- **Fase 3: Triage ed elaborazione mirata della pagina.** Sulla base dell'analisi precedente, ogni singola pagina subisce il processo di smistamento intelligente mostrato nella parte inferiore della Figura 4.1, venendo indirizzata allo strumento di estrazione più adeguato:
 - Se la pagina presenta un layout lineare e un'elevata densità di testo nativo, viene classificata come *Semplice* e affidata al processo di Open Data Loader.
 - Se la pagina contiene elementi geometrici articolati o tabelle annidate, ma il testo rimane comunque selezionabile, viene classificata come *Complessa* e processata tramite Docling Standard (senza abilitare l'OCR).

- Se la pagina è priva di testo nativo (es. una fotocopia), viene classificata come *Scansione* e inviata al container specializzato di Docling, dove interviene il motore RapidOCR per il riconoscimento ottico dei caratteri.

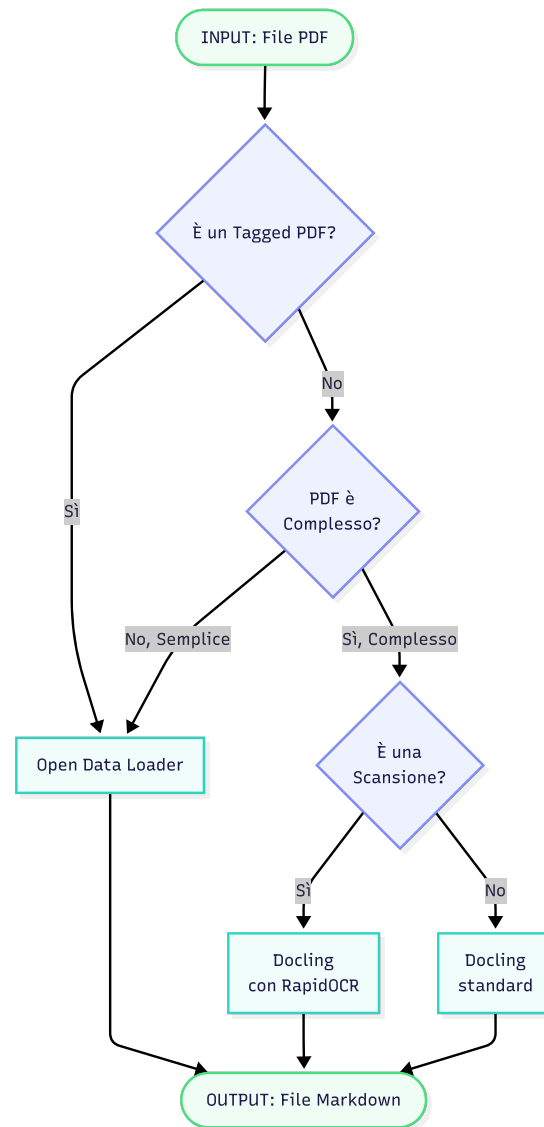


Figura 4.1: Flowchart del Custom Triage Processor

4.2 Strutturazione dei Documenti tramite LLM

Una volta ultimata la fase di estrazione del testo grezzo, illustrata nella sezione precedente, i documenti normativi si presentano sotto forma di file testuali piatti in formato Markdown. Tali file, pur contenendo l'intero testo originale, sono del tutto privi di metadati strutturali e di una formattazione gerarchica chiara. Poiché l'architettura di sistema prevede l'inserimento di questi documenti all'interno di Neo4j risulta indispensabile trasformare il testo non strutturato in una gerarchia rigorosa. L'archiviazione del solo testo grezzo, infatti, risulterebbe del tutto inadeguata e priva di utilità pratica. Una strutturazione accurata rappresenta il prerequisito fondamentale per abilitare qualsiasi operazione avanzata di *information retrieval*, garantendo che i dati possano essere interrogati in modo efficiente e contestuale.

4.2.1 Il Modello e la Strategia di Generazione

Per affrontare il compito cruciale della strutturazione sintattica, è stato impiegato il Large Language Model Gemini 3.5 Flash. A livello di architettura software, il processo è diviso in due step distinti: la generazione di una struttura JSON gerarchica e l'estrazione delle relazioni semantiche tra le entità legali.

L'efficacia e l'accuratezza del processo di strutturazione si fondano su un *System Prompt* ingegnerizzato ad hoc. Il suo compito principale è quello di mappare il contenuto all'interno del seguente schema logico:

- **Metadati (metadata):** Il modello estrae e categorizza i dati identificativi dell'atto. Vengono isolati campi chiave come la data di stipula, il titolo completo e il nome del file originale.
- **Sezioni Introduttive e Conclusive:** Vengono separate dal corpo normativo tutte le sezioni di contorno. Tra queste figurano le premesse iniziali e le epigrafi (preface), gli indici (preamble), le dichiarazioni di chiusura con le relative firme (conclusions) e gli eventuali documenti annessi (attachments).
- **Corpo del Documento (body):** Rappresenta il nucleo dell'algoritmo di strutturazione. Il modello deve nidificare il testo seguendo fedelmente la gerarchia complessa e rigida tipica degli atti normativi italiani: **Titoli > Capi > Articoli > Commi**.

Per garantire la massima coerenza dei dati all'interno del Knowledge Graph, sono state imposte al LLM regole tassative di parsing:

- **Normalizzazione Numerica:** Tutti gli identificativi gerarchici originariamente espressi in numeri romani o in forma testuale (es. "TITOLO I", "CAPO IV") devono essere categoricamente convertiti in numeri interi arabi ("TITOLO 1", "CAPO 4").

- **Gestione Dinamica dei Commi:** Poiché nei testi di legge i commi non sono sempre esplicitamente numerati, il modello è istruito a dedurli analizzando le interruzioni logiche dei paragrafi, assegnando loro una numerazione progressiva.
- **Ottimizzazione dello Schema:** Per mantenere leggeri i file JSON, il modello è programmato per non inizializzare chiavi con valori vuoti o nulli; se un livello gerarchico (come un capo o un titolo) non trova riscontro nel testo originale, la proprietà viene semplicemente omessa, fermo restando che negli atti normativi i livelli fondamentali come articoli e commi esistono necessariamente.

L'esecuzione di questo processo restituisce una directory di documenti JSON puliti e strutturati gerarchicamente, pronti per affrontare il successivo step: l'estrazione delle relazioni semantiche tra le entità legali.

4.2.2 Estrazione delle Relazioni Semantiche

La sola scomposizione del testo normativo in una struttura gerarchica ad albero rappresenta una condizione necessaria ma non sufficiente per la costruzione di un vero e proprio *Knowledge Graph*. Per sfruttare appieno la potenza topologica di un database a grafo come Neo4j, è indispensabile individuare e mappare esplicitamente i collegamenti e le dipendenze logiche che intercorrono tra le diverse disposizioni giuridiche all'interno dello stesso atto. A questo scopo, la pipeline prevede un secondo step logico interamente dedicato all'estrazione delle relazioni semantiche. In questa fase, l'input non è più il testo grezzo, ma i file JSON prodotti nello step precedente, che quindi agiscono come una base di conoscenza consolidata. Ciascun documento viene quindi analizzato dal LLM Gemini 3.5 Flash, guidato da un *System Prompt* specializzato. Tale prompt è stato progettato per riconoscere i costrutti linguistici del gergo giuridico che indicano un collegamento formale tra le parti del testo. Le relazioni estratte sono:

- **CITA:** Un articolo o comma fa riferimento a un'altra disposizione.
- **ABROGA:** Una disposizione annulla la validità giuridica di un'altra.
- **DEROGA:** Un elemento introduce un'eccezione a una regola generale.
- **SOSTITUISCE:** Una disposizione prende il posto di un'altra.
- **INTERPRETA:** La comprensione di una norma dipende dalla lettura di un'altra.

L'output di questa analisi è una rete di collegamenti formali, dove per ciascuna relazione vengono specificati il nodo di partenza, il nodo di arrivo e il tipo di legame. Queste informazioni sono esportate in un nuovo set di file JSON.

Questo approccio a due stadi permette di separare le responsabilità e ridurre il carico cognitivo del modello a ogni ciclo. In questo modo si produce un dataset pulito e affidabile, pronto per essere caricato nel grafo.

4.3 Importazione dati nei Database

Una volta completata la strutturazione dei documenti legali in formato JSON, si apre la fase di persistenza e indicizzazione dei dati. Nel contesto di questo progetto di tirocinio, l'architettura prevede il caricamento dei dati estratti all'interno del database a grafo Neo4j.

Per garantire la flessibilità e favorire un ambiente di sviluppo pulito e modulare, lo sviluppo del software ha visto l'applicazione del *Repository Pattern* e l'uso di interfacce polimorfiche.

4.3.1 I Modelli di Dominio

Prima di analizzare le strategie di persistenza, è fondamentale descrivere come i dati estratti vengano rappresentati. All'interno della cartella `models` di questo step, il sistema definisce le entità fondamentali del dominio normativo utilizzando il paradigma ad oggetti. Queste classi operano essenzialmente come *Entities*: strutture dati pure, prive di logica di persistenza diretta o dipendenze verso i DBMS, il cui unico scopo è incapsulare i dati del dominio.

L'architettura dei modelli è rappresentata nel diagramma UML della Figura 4.2. Le principali classi sono:

- **Entity**: Incapsula l'ente pubblico emittente, memorizzando i metadati di dominio.
- **LegalAct**: È la classe orchestratrice principale. Rappresenta l'atto legale nella sua interezza e contiene i metadati principali (`title`, `publication_date`, `file_name`) e soprattutto una lista `elements` che tiene traccia di tutti i sotto-elementi istanziati e delle loro relazioni gerarchiche.
- **StructuralElement**: È la classe base polimorfica per mappare le sottoclassi `Title`, `Chapter` e `Article`, mantenendo traccia del numero d'ordine e dell'eventuale rubrica.
- **Paragraph**: Rappresenta il comma ed è l'unità atomica d'informazione sulla quale verranno calcolati gli `embedding`.

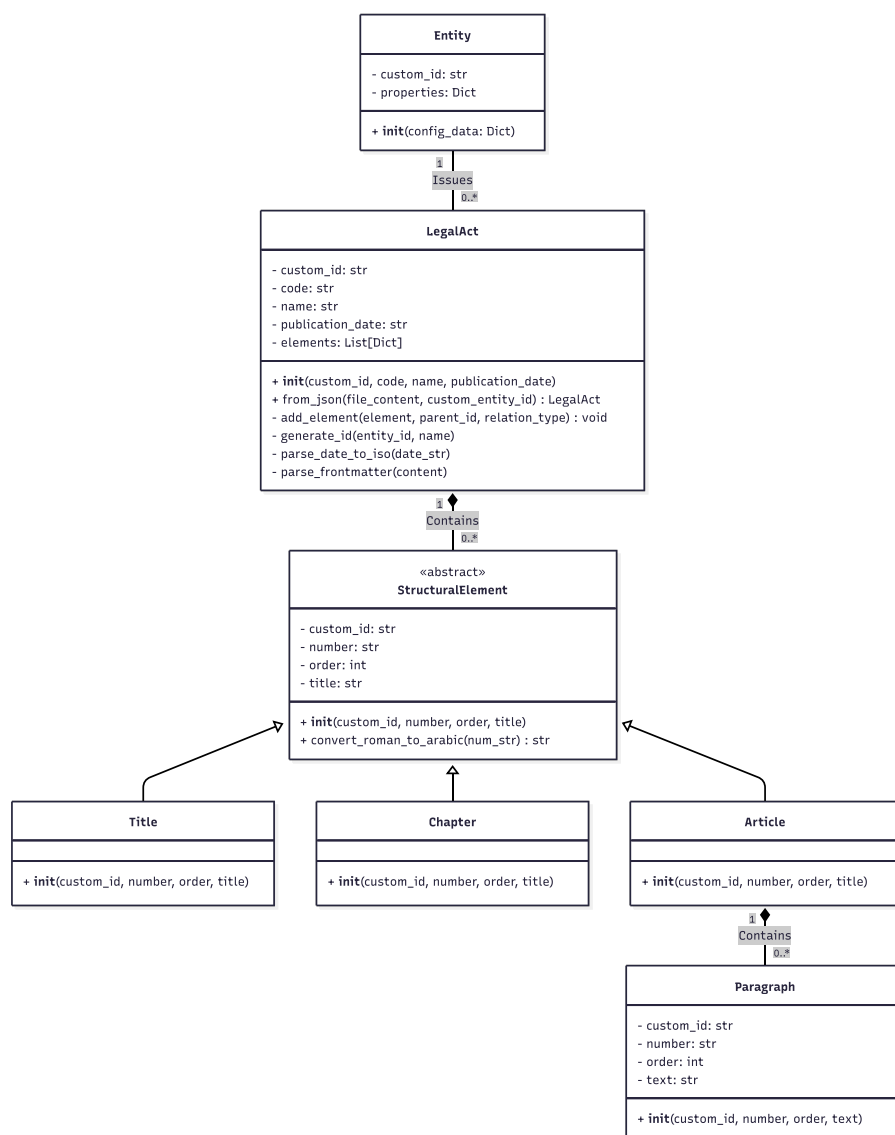


Figura 4.2: Class Diagram dei modelli di dominio

4.3.2 Interfaccia per il disaccoppiamento della persistenza

L'adozione di un'interfaccia comune di importazione nasce dalla precisa volontà di disaccoppiare la logica di lettura dei dati dalla loro effettiva persistenza fisica. Sebbene l'attuale implementazione preveda l'utilizzo esclusivo del database a grafo Neo4j, questa scelta architetturale si rivela strategica per sviluppi futuri. Qualora si rendesse necessario integrare nuovi DBMS (ad esempio, un database puramente vettoriale dedicato esclusivamente agli embedding), sarà sufficiente implementare una nuova classe concreta senza dover alterare il flusso principale del programma. Inoltre, il disaccoppiamento garantisce un'elevata testabilità del codice, permettendo di validare la logica di dominio e il parsing

dei documenti simulando le operazioni di salvataggio indipendentemente dal database sottostante.

Nel sistema sviluppato, l'interfaccia astratta di riferimento è rappresentata dalla classe `DBImporterInterface`. Essa definisce tre metodi fondamentali che ogni classe concreta deve implementare:

- `connect()`: per la gestione e la validazione della connessione con l'istanza specifica del database;
- `import_legal_act(entity, legal_act)`: per l'esecuzione effettiva di ingestion dell'atto normativo e dell'ente emittente;
- `close()`: per la chiusura sicura delle sessioni.

4.3.3 Architettura del Database Neo4j

Il database modella fedelmente l'albero gerarchico discendente del documento, garantendo la flessibilità necessaria per gestire percorsi diretti qualora i livelli intermedi siano stati omessi durante la strutturazione JSON (es. `LegalAct` → `Article` → `Paragraph`).

Le *Label* dei nodi sul grafo (ovvero **Entity**, **LegalAct**, **Title**, **Chapter**, **Article** e **Paragraph**) riflettono rigorosamente la struttura delle classi di dominio già presentate nella Sezione 4.3.1, così proprio come le *Properties* dei nodi, che conservano i metadati principali e le informazioni testuali.

Un aspetto architetturale chiave, comune a tutti i nodi, è la presenza della proprietà `custom_id`. Tale attributo codifica per intero il percorso gerarchico del nodo all'interno del documento (ad esempio, `ente__atto__title-1__chapter-2__article-3__paragraph-4`). Grazie a questo approccio, il database può sfruttare l'indicizzazione su questa proprietà per individuare immediatamente un singolo nodo, azzerando le operazioni di attraversamento del grafo e garantendo un recupero delle informazioni in tempo costante.

All'interno di questa topologia, i nodi di tipo **Paragraph** ricoprono un ruolo centrale per le funzionalità avanzate del sistema. Oltre a conservare le proprietà testuali, in questi nodi vengono salvate le rappresentazioni vettoriali dei dati: in particolare, gli embedding semantici a dimensione variabile e un embedding strutturale a 128 dimensioni calcolato mediante l'algoritmo di *Node2Vec* (maggiori informazioni sono nel capitolo 4.4). L'archiviazione di tali vettori è progettata per abilitare le successive operazioni di interrogazione.

L'ecosistema di relazioni (*Edges*) all'interno del grafo si articola in tre macro-categorie progettate per supportare le interrogazioni avanzate:

- **Relazioni Strutturali**: Definiscono la topologia discendente del documento normativo. Comprendono l'arco `ISSUES` che lega l'Ente all'Atto, e la catena di contenimento gerarchico formata da `HAS_TITLE`, `HAS_CHAPTER`, `HAS_ARTICLE` e `HAS_PARAGRAPH`.

- **Relazioni Sequenziali:** I commi appartenenti allo stesso articolo sono concatenati linearmente tramite la relazione NEXT_PARAGRAPH. Questa architettura a lista concatenata si rivela fondamentale per espandere dinamicamente la finestra di contesto durante la fase di interrogazione, permettendo di recuperare i commi logicamente antecedenti o successivi senza dover eseguire query computazionalmente costose per ricalcolare l'ordine strutturale.
- **Relazioni Legali:** Traducono in archi orientati sul grafo i collegamenti logici e giuridici intra-documento individuati dal prompt specializzato dell'LLM (Sezione 4.2). Le cinque tipologie di relazione estratte vengono trasposte fedelmente nelle label CITES, REPEALS, INTERPRETS, REPLACES e DEROGATES.

Occorre precisare che, sebbene lo schema dati supporti nativamente tutte le interazioni legali sopracitate, l'attuale popolamento del database non registra occorrenze per le relazioni REPEALS e REPLACES. Questa mancanza non è imputabile a un difetto del sistema di estrazione, bensì rappresenta un limite intrinseco dei documenti elaborati, che non contengono clausole esplicite di abrogazione o sostituzione strutturata.

4.3.4 Dettagli Implementativi dell'Ingestion su Neo4j

Un atto legale non può essere salvato tramite un'unica operazione atomica standard, a causa delle complesse connessioni tra i nodi, per questo motivo la responsabilità è stata distribuita su una serie di repository specializzati, ognuno deputato alla gestione del ciclo di vita di un singolo modello di dominio:

- **EntityRepository:** Gestisce l'inserimento dei nodi relativi agli enti istituzionali.
- **LegalActRepository:** Inserisce il nodo radice dell'atto e lo ancora all'ente emittente.
- **StructuralElementRepository:** Incapsula la logica Cypher necessaria a creare i nodi intermedi del grafo e gli archi verso i rispettivi nodi padri.
- **ParagraphRepository:** Si occupa esclusivamente della persistenza dei nodi foglia contenenti il testo dei commi.

Per quanto riguarda il salvataggio delle relazioni trasversali, queste informazioni vengono lette da file JSON esterni dedicati (Sezione 4.2.2), per poi essere elaborate da un metodo che, attraverso i `custom_id` dei nodi salvati e indicizzati sul grafo, trova in maniera quasi istantanea i nodi correlati dall'estratto JSON ed esegue una query Cypher dedicata per la creazione dell'arco semantico.

Listing 4.1 Query Cypher per l'inserimento delle relazioni legali

```
1: MATCH (s {custom_id: $source_id})
2: WITH s
```

```

3: MATCH (t {custom_id: $target_id})
4: MERGE (s)-[r:'<rel_type>']->(t)
5: RETURN r

```

L'obiettivo di questa query è rintracciare i due nodi di interesse (sorgente e destinazione) tramite il loro identificativo univoco e stabilire una relazione orientata tra di essi.

Di seguito troviamo un esempio di relazione estratta in formato JSON, con ID parzialmente omessi per motivi di leggibilità:

Listing 4.2 Esempio reale di relazione estratta in formato JSON

```

1: {
2:   "sourceId": "ente-provincia-taranto__contratto-collettivo
   -[...]__title-2__chapter-2__article-8__paragraph-6",
3:   "relationshipType": "CITES",
4:   "targetId": "ente-provincia-taranto__contratto-collettivo
   -[...]__title-2__chapter-2__article-8__paragraph-1"
5: }

```

A supporto di quanto descritto, la Figura 4.3 fornisce una rappresentazione visiva delle relazioni instaurate tra i nodi dell'articolo 8 del seguente atto: "CCNL del personale non dirigente del comparto regioni e autonomie locali"

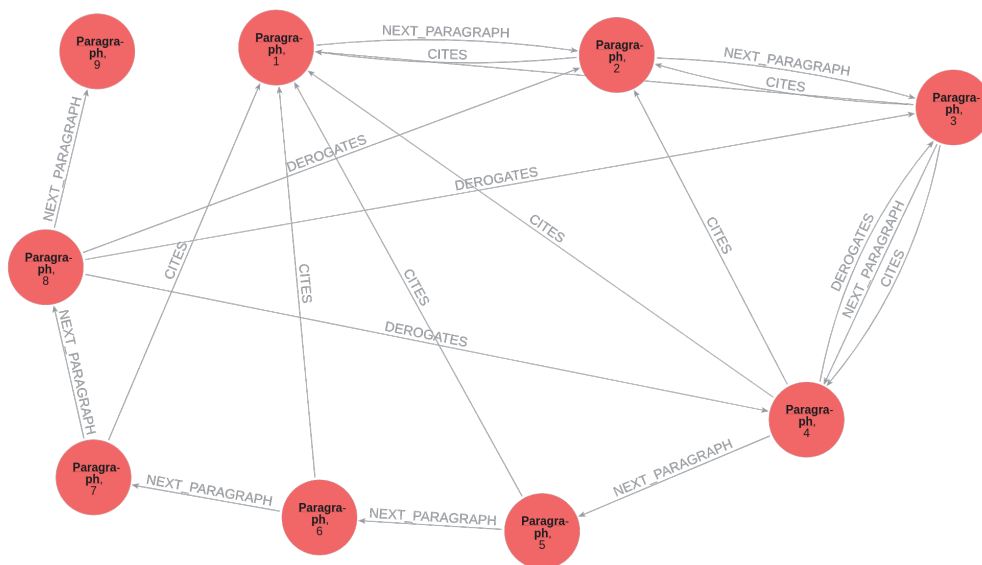


Figura 4.3: Rappresentazione visiva delle relazioni instaurate tra i nodi dell'articolo 8 di un atto normativo.

4.3.5 Risultati e Metriche

L'esecuzione della pipeline di ingestion ha dimostrato un'elevata efficienza, completando il caricamento dell'intero dataset all'interno del database in un tempo complessivo di **50.29 secondi**.

Il processo ha popolato il grafo con un totale di 8.535 nodi e 14.201 relazioni. Di seguito viene riportato il dettaglio quantitativo delle entità di dominio e degli archi salvati.

La Tabella 4.1 illustra la distribuzione dei nodi all'interno del grafo, raggruppati per *Label*. Come si può osservare dai dati, vi è una netta preponderanza numerica dei nodi Paragraph, i quali rappresentano l'unità atomica d'informazione su cui si basano le successive operazioni di embedding e ricerca.

Label Nodo	Quantità
Paragraph	6.396
Article	1.802
Chapter	147
LegalAct	95
Title	93
Entity	2

Tabella 4.1: Statistica dei nodi inseriti nel database Neo4j.

Per quanto riguarda invece le interconnessioni tra le entità, la Tabella 4.2 dettaglia il conteggio esatto degli archi strutturali, sequenziali e legali creati durante l'importazione.

Tipo di Relazione	Quantità
HAS_PARAGRAPH	6.396
NEXT_PARAGRAPH	4.594
HAS_ARTICLE	1.802
CITES	1.030
HAS_CHAPTER	147
ISSUES	95
HAS_TITLE	93
DEROGATES	41
INTERPRETS	3

Tabella 4.2: Statistica delle relazioni inserite nel database Neo4j.

Come si evince dalla Tabella 4.2, l'analisi degli archi conferma la solidità dell'architettura a lista concatenata, evidenziata dal notevole volume di relazioni NEXT_PARAGRAPH (4.594), essenziali per l'espansione dinamica del contesto in fase di retrieval.

Inoltre, analizzando sempre i medesimi dati in merito agli archi logico-legali, si osserva una netta predominanza delle relazioni di citazione (CITES, 1.030 occorrenze), seguite da un numero minoritario di deroghe (DEROGATES, 41) e interpretazioni (INTERPRETS, 3). Coerentemente con quanto anticipato nella Sezione 4.3.3, le metriche riportate confermano l'assenza totale di occorrenze per le relazioni di abrogazione (REPEALS) e sostituzione (REPLACES) all'interno di questo specifico bacino documentale.

4.3.6 Visualizzazione del Grafo

A conclusione di questo capitolo, si propone una rappresentazione visiva di un estratto del database Neo4j volta a mostrare in maniera intuitiva la topologia della rete. Nella Figura 4.4 è possibile osservare la struttura gerarchica e le connessioni tra le diverse entità, evidenziate mediante una differenziazione cromatica:

- I **nodi gialli** rappresentano l'ente emittente (Entity);
- I **nodi azzurri** indicano gli atti legali (LegalAct);
- I **nodi rossi** corrispondono ai commi (Paragraph);
- I **restanti nodi** raffigurano le altre parti della gerarchia strutturale del documento, ossia i titoli (Title), i capi (Chapter) e gli articoli (Article).

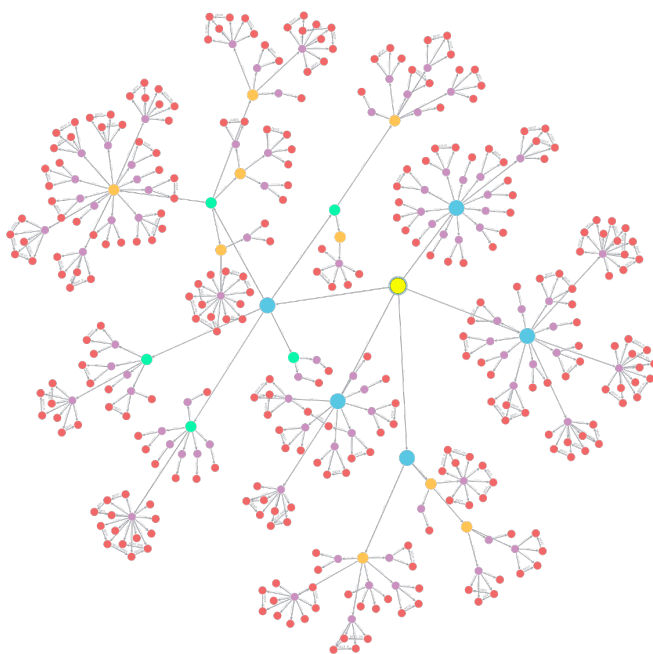


Figura 4.4: Rappresentazione visiva di un estratto del database a grafo Neo4j.

4.4 Creazione degli Embedding

L'operazione di vettorizzazione, o di *embedding*, rappresenta un passaggio cruciale per abilitare le funzionalità di ricerca semantica all'interno del database a grafo. La pipeline supporta tre modelli di embedding, dettagliati nella sezione 2.4:

- google/embeddinggemma-300m
- perplexity-ai/pplx-embed-v1-0.6b
- perplexity-ai/pplx-embed-context-v1-0.6b

I vettori vengono archiviati direttamente come proprietà all'interno dei nodi (:Paragraph) già esistenti nel grafo. Questo evita la necessità di un database vettoriale separato e consente al modello di ricerca di operare direttamente su Neo4j.

Neo4j, inoltre, supporta la creazione di indici vettoriali utilizzando algoritmi di ricerca approssimata (*Approximate Nearest Neighbor Search*, ANN). Invece di calcolare la distanza coseno tra il vettore di query e tutti gli altri vettori (operazione $O(n)$ molto costosa), questi indici riducono drasticamente lo spazio di ricerca. Di conseguenza, il tempo di esecuzione delle query passa da secondi a millisecondi, rendendo la ricerca praticabile anche su documenti normativi di grandi volumi.

4.4.1 Differenza nell'Arricchimento Contestuale: Standard vs Late Chunking

Il sistema implementa due approcci diversi per fornire *contesto* ai commi. Se si fornisce al modello solo la stringa del comma «... è punito con la reclusione da 1 a 3 anni», l'embedding generato risulterà limitato, poiché il testo manca di informazioni fondamentali come il soggetto e il reato specifico (elementi definiti nei livelli superiori come l'articolo o il titolo). Ecco come le due metodologie risolvono il problema:

A. Arricchimento Standard (Modelli: Gemma 300m e PPLX-v1)

In questo approccio, il contesto viene iniettato testualmente attraverso *String Concatenation*. Per ciascun comma, viene recuperato il contesto gerarchico risalendo l'albero strutturale tramite query Cypher. Tutte queste informazioni vengono concatenate in un'unica stringa, producendo un testo inviato al modello simile al seguente:

```
«Atto: Codice Penale | Titolo 3 Dei delitti contro l'amministrazione  
della giustizia | Capo 2 Dei delitti contro l'autorità delle  
decisioni giudiziarie | Art. 385 Evasione | Comma 1 | ... è punito  
con la reclusione da 1 a 3 anni.»
```

In questo approccio, ogni comma viene trattato come un *chunk isolato* al momento dell'embedding. Sebbene il modello riceva il testo arricchito dai metadati gerarchici, il contesto rimane limitato a questo preambolo testuale statico.

B. Arricchimento tramite Late Chunking (Modello: PPLX-Context)

Questo approccio impiega una vera e propria contestualizzazione a livello di rete neurale. A differenza dell'approccio standard, non utilizza preamboli testuali pre-calcolati, ma affida la comprensione semantica direttamente all'algoritmo di *self-attention* del modello.

Il sistema recupera dal database, tramite query Cypher, l'intero articolo con tutti i commi che lo compongono, mantenendo traccia dell'ordine sequenziale e dei confini esatti tra di essi. Invece di frammentare il testo a monte e inviare al modello i commi singolarmente, il sistema li presenta all'encoder come un'unica sequenza di input ininterrotta.

È in questa fase che agisce il meccanismo di *late chunking*. Il modello processa l'articolo nella sua interezza attraverso i successivi layer del Transformer; in questo modo, la *self-attention* bidirezionale permette ai token di ciascun comma di "prestare attenzione" e interagire con i token adiacenti e con l'intero testo. Questo approccio abbatte i confini artificiali del chunking testuale, permettendo alla rete di risolvere dinamicamente pronomi, riferimenti incrociati e concetti impliciti basandosi sul contesto normativo globale.

Solo nella fase finale, dopo che ogni singolo token ha assorbito un significato profondamente contestualizzato, il modello applica un'operazione di *pooling* differenziata. Sfruttando gli indici sequenziali salvati in precedenza, estrae e restituisce gli embeddings separati per ciascun comma originale. Questa architettura garantisce che l'embedding finale di un singolo comma racchiuda non solo il suo significato letterale, ma anche le sfumature semantiche derivanti dal resto dell'articolo, unendo la massima precisione granulare a una perfetta consapevolezza del contesto globale.

Un esempio pratico:

Comma 1: "I soggetti che omettono la dichiarazione dei redditi sono puniti con una sanzione di 1.000 euro."

Comma 2: "La disposizione non si applica se essi dimostrano l'assenza di dolo."

Se il modello processa il Comma 2 in isolamento, la parola "essi" è ambigua e "La disposizione" è un concetto vuoto. Sfruttando il late chunking, la rete neurale collega istantaneamente "essi" del Comma 2 ai "soggetti" del Comma 1, e "La disposizione" all'intero concetto di sanzione appena letto.

4.4.2 Il Graph Embedding come appoggio agli Embedding Semantici

Oltre all'analisi puramente testuale, il sistema arricchisce la rappresentazione della conoscenza integrando l'informazione strutturale del documento. A tale scopo viene impiegato l'algoritmo **Node2Vec**, eseguito tramite la libreria Graph Data Science (GDS). Questo approccio genera un *graph embedding* per ogni comma, codificando in vettori a 128 dimensioni la topologia della rete, ovvero la gerarchia del documento e le relazioni logico-giuridiche che legano i nodi. In questo modo, l'algoritmo mappa in regioni vicine dello

spazio vettoriale quei commi che, a prescindere dal testo, condividono una medesima posizione strutturale o forti vincoli normativi. Per adattare Node2Vec al dominio legale, la strategia di esplorazione del grafo tramite *random walk* è stata calibrata attraverso i parametri **p** e **q**:

- **Return parameter** ($p = 1$): Mantiene un comportamento esplorativo neutrale, senza forzare né penalizzare il ritorno immediato al nodo precedente.
- **In-out parameter** ($q = 0.5$): Essendo inferiore a 1, forza un'esplorazione profonda (simile a una *Depth-First Search*). Questa scelta è essenziale nel dominio legale, in quanto permette all'algoritmo di attraversare in profondità l'intera struttura verticale del documento (Legge → Titolo → Capo → Articolo → Comma) e di seguire agevolmente lunghe catene di rimandi normativi.

Inoltre, per guidare attivamente l'apprendimento verso i percorsi più significativi, è stato implementato un sistema di pesi differenziati per le diverse tipologie di relazione. Questi valori alterano le probabilità di transizione durante le passeggiate aleatorie, forzando il modello a privilegiare specifiche tipologie di relazioni:

- **Pesi Elevati (2.0)**: Assegnati alle relazioni strutturali (HAS_TITLE, HAS_CHAPTER, ecc.) per preservare la forte dipendenza gerarchica di un comma dal suo contenitore genitore. Questo stesso valore massimo è stato riservato ad abrogazioni e sostituzioni (REPEALS, REPLACES), in quanto rappresentano legami di importanza cruciale: tali relazioni, infatti, hanno un impatto giuridico estremamente forte, andando ad alterare in modo diretto la validità e l'efficacia delle disposizioni normative connesse.
- **Pesi Intermedi (1.5)**: Assegnati a interpretazioni e deroghe (INTERPRETS, DEROGATES), relazioni che modificano l'applicazione di una norma creando vincoli forti, ma senza costituire una sostituzione assoluta.
- **Pesi Base (1.0)**: Assegnati alla contiguità testuale (NEXT_PARAGRAPH) e alle citazioni generiche (CITES), collegamenti sicuramente pertinenti ma gerarchicamente e semanticamente meno vincolanti.

Al termine dell'esecuzione, questi embedding strutturali vengono salvati come proprietà aggiuntiva (`embedding_node2vec_128`) sui nodi dei commi (`:Paragraph`). Questo affianca l'embedding semantico tradizionale, dotando il sistema di un duplice livello di ricerca: uno basato sul contenuto linguistico, l'altro sulla topologia e sui legami normativi del documento.

Bibliografia

- [1] *Neo4j Graph Platform*. URL: <https://neo4j.com>.
- [2] *Python Language Reference*. URL: <https://www.python.org>.
- [3] *Open Data Loader*. URL: <https://github.com/opendataloader-project/opendataloader-pdf>.
- [4] *Docling: Document Parsing and Layout Analysis*. URL: <https://github.com/DS4SD/docling>.
- [5] *RapidOCR: OCR Engine*. URL: <https://github.com/RapidAI/RapidOCR>.
- [6] *Cypher Query Language Reference*. URL: <https://neo4j.com/docs/cypher-manual>.
- [7] Neo4j, Inc. *Neo4j Graph Data Science Library*. <https://neo4j.com/docs/graph-data-science/current/>. 2024.
- [8] Aditya Grover e Jure Leskovec. «Node2vec: Scalable feature learning for networks». In: *Proceedings of the 22nd ACM SIGKDD international conference on Knowledge discovery and data mining* (2016).
- [9] *Gemini 3.5 Flash Model Card*. URL: <https://storage.googleapis.com/deepmind-media/Model-Cards/Gemini-3-5-Flash-Model-Card.pdf>.
- [10] Michael Günther et al. *Late Chunking: Contextual Chunk Embeddings Using Long-Context Embedding Models*. 2024. arXiv: 2409.04701 [cs.IR]. URL: <https://arxiv.org/abs/2409.04701>.
- [11] *Docker: Containerization Platform*. URL: <https://www.docker.com>.